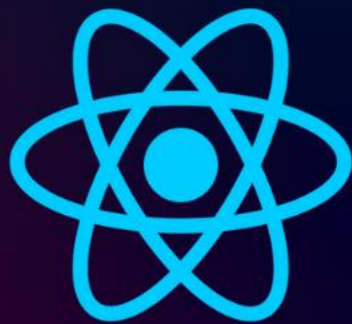


[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.1

Refs & Portals

Giới thiệu dự án
ôn tập useState



<http://Tuhoc.CC>



```
<>
  <Header />
  <Player />
  <div id="challenges"></div>
</>
```

src > components > Header.jsx > ...

```
1 export default function Header() {
2   return (
3     <header>
4       <h1>
5         Đếm Ngược <em>Căn Thời Gian</em>!
6       </h1>
7       <p> Thử tài ước lượng thời gian </p>
8     </header>
9   );
10 }
```

ĐẾM NGƯỢC CĂN THỜI GIAN!

Thử tài ước lượng thời gian

Welcome Tuhoc.CC

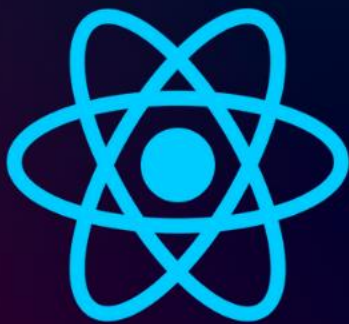
Set Name

src > components > Player.jsx > ...

```
1 import {useState} from "react";
2
3 export default function Player() {
4   const [enteredPlayerName, setEnteredPlayerName] = useState(null);
5   const [submitted, setSubmitted] = useState(false);
6
7   function handleChange(event) {
8     setSubmitted(false);
9     setEnteredPlayerName(event.target.value);
10  }
11
12  function handleClick() {
13    setSubmitted(true);
14  }
15  return (
16    <section id="player">
17      <h2>Welcome {submitted ? enteredPlayerName : "No Name"} </h2>
18      <div>
19        <input type="text" onChange={handleChange} value={enteredPlayerName} />
20        <button onClick={handleClick}>Set Name</button>
21      </div>
22    </section>
23  );
24 }
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.2

Refs & Portals

React useRef



<http://Tuhoc.CC>



2

useRef

```
import {useState, useRef} from "react";

export default function Player() {
  const playerName = useRef();

  const [enteredPlayerName, setEnteredPlayerName] = useState(null);

  function handleClick() {
    setEnteredPlayerName(playerName.current.value);
  }

  return (
    <section id="player">
      <h2>Welcome {enteredPlayerName ?? "No Name"} </h2>
      <p>
        <input ref={playerName} type="text" />
        <button onClick={handleClick}>Set Name</button>
      </p>
    </section>
  );
}
```

- ✓ *useRef* là một hook đặc biệt trong React, dùng để tạo ra một đối tượng tham chiếu (reference object).
- ✓ *useRef()* trả về một object có property *.current*
- ✓ Khi gán thuộc tính *ref={playerName}* (ví dụ: *<input ref={playerName} />*), React sẽ tự động gán element DOM thực tế vào *playerName.current*

2

useRef

```
<h2>Welcome {enteredPlayerName ? enteredPlayerName : "No Name"} </h2>
```

```
<h2>Welcome {enteredPlayerName ?? "No Name"} </h2>
```

✓ ***$a ?? b$** : Nếu a khác null và khác undefined, trả về a , còn lại trả về b*

```
let x = null;
let y = x ?? "default"; // y = "default"

let x2 = '';
let y2 = x2 ?? "default"; // y2 = '' (vẫn lấy giá trị x2)
```

***?** : kiểm tra tất cả giá trị falsy.*

***??** chỉ kiểm tra null và undefined*

❏ Kiến thức liên quan : <http://js.tuhoc.cc>

34



18. Truthy and Falsy Values -
Chuyển đổi kiểu dữ liệu với...
Gà Lại Lập Trình

2

useRef

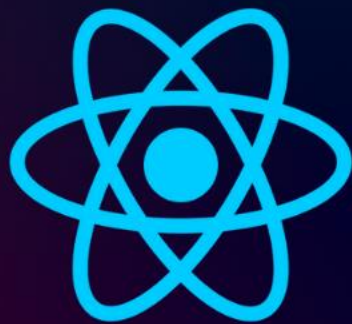
✓ So sánh code khi dùng *useState* và *useRef*

```
Player.jsx x
src > components > Player.jsx > Player
1 import {useState} from "react";
2
3 export default function Player() {
4   const [enteredPlayerName, setEnteredPlayerName] = useState(null);
5   const [submitted, setSubmitted] = useState(false);
6
7   function handleChange(event) {
8     setSubmitted(false);
9     setEnteredPlayerName(event.target.value);
10  }
11
12  function handleClick() {
13    setSubmitted(true);
14  }
15  return (
16    <section id="player">
17      <h2>Welcome {submitted ? enteredPlayerName : "No Name"} </h2>
18      <p>
19        <input type="text" onChange={handleChange} value={enteredPlayerName} />
20        <button onClick={handleClick}>Set Name</button>
21      </p>
22    </section>
23  );
24 }
25
```

```
Player.jsx x
src > components > Player.jsx > Player
1 import {useState, useRef} from "react";
2
3 export default function Player() {
4   const playerName = useRef();
5   const [enteredPlayerName, setEnteredPlayerName] = useState(null);
6
7   function handleClick() {
8     setEnteredPlayerName(playerName.current.value);
9   }
10  return (
11    <section id="player">
12      <h2>Welcome {enteredPlayerName ?? "No Name"} </h2>
13      <p>
14        <input ref={playerName} type="text" />
15        <button onClick={handleClick}>Set Name</button>
16      </p>
17    </section>
18  );
19 }
20
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.3

Refs & Portals

useRef và useState ?



<http://Tuhoc.CC>



3

useRef vs useState ?

✓ So sánh code khi dùng useState và useRef

```
Player.jsx
src > components > Player.jsx > Player
1 import {useState} from "react";
2
3 export default function Player() {
4   const [enteredPlayerName, setEnteredPlayerName] = useState(null);
5   const [submitted, setSubmitted] = useState(false);
6
7   function handleChange(event) {
8     setSubmitted(false);
9     setEnteredPlayerName(event.target.value);
10  }
11
12  function handleClick() {
13    setSubmitted(true);
14  }
15  return (
16    <section id="player">
17      <h2>Welcome {submitted ? enteredPlayerName : "No Name"} </h2>
18      <p>
19        <input type="text" onChange={handleChange} value={enteredPlayerName} />
20        <button onClick={handleClick}>Set Name</button>
21      </p>
22    </section>
23  );
24 }
25
```

```
Player.jsx
import {useState, useRef} from "react";
1
export default function Player() {
2   const playerName = useRef();
3
4   const [enteredPlayerName, setEnteredPlayerName] = useState(null);
5
6   function handleClick() {
7     setEnteredPlayerName(playerName.current.value);
8   }
9   return (
10     <section id="player">
11       <h2>Welcome {enteredPlayerName ?? "No Name"} </h2>
12       <p>
13         <input ref={playerName} type="text" />
14         <button onClick={handleClick}>Set Name</button>
15       </p>
16     </section>
17   );
18 }
```

3

useRef vs useState ?

✓ *Câu hỏi: Nếu dùng useRef có thể tham chiếu đến đối tượng, thì tại sao cần useState ?*

```
Player.jsx 1 X
src > components > Player.jsx > ...

3 export default function Player() {
4   const playerName = useRef();
5
6   // const [enteredPlayerName, setEnteredPlayerName] = useState(null);
7
8   function handleClick() {
9     // setEnteredPlayerName(playerName.current.value);
10    console.log(playerName.current.value); // Đọc giá trị trong input
11  }
12  return (
13    <section id="player">
14      {/* <h2>Welcome {enteredPlayerName ?? "No Name"} </h2> */}
15      <h2>Welcome {playerName.current.value ?? "No Name"} </h2>
16      <p>
17        <input ref={playerName} type="text" />
18        <button onClick={handleClick}>Set Name</button>
19      </p>
20    </section>
21  );
22 }
```

✗ ▶ Uncaught TypeError: Cannot read properties of undefined (reading 'value')
at Player (Player.jsx:15:40)
[Show ignore-listed frames](#)

3

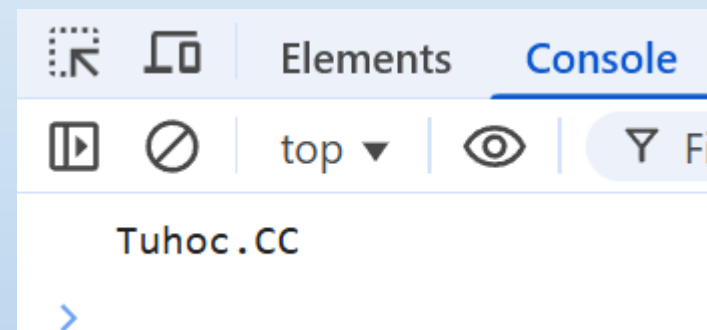
useRef vs useState ?

- ✓ Khi khởi tạo lần đầu tiên, *playerName.current.value* chưa có giá trị, nó đang là *undefine*

```
{<h2>Welcome {playerName.current.value ?? "No Name"} </h2>}
```

- ✓ Chúng ta có thể giải quyết bằng toán tử 3 ngôi

```
{<h2>Welcome {playerName.current ? playerName.current.value : "No Name"} </h2>}
```

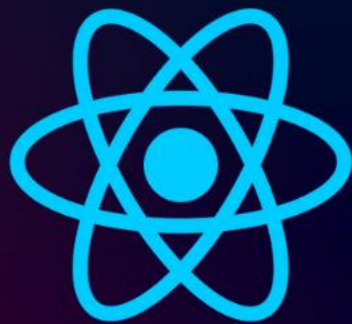


Trình duyệt đã không còn báo lỗi, nhưng khi nhấn Set Name thì không có tác dụng

Hạng mục	State (Trạng thái)	Refs (Tham chiếu)
Ảnh hưởng đến UI	Có. Khi thay đổi, giao diện sẽ cập nhật lại (re-render).	Không. Thay đổi ref không làm giao diện cập nhật lại.

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.4

Refs & Portals

Add Component Timer



<http://Tuhoc.CC>



4

Add Component Timer

components

- Header.jsx
- Player.jsx
- TimeStopper.jsx**

```
TimeStopper.jsx x App.jsx
src > components > TimeStopper.jsx > ...
1 export default function TimeStopper({title, targetTime}) {
2   return (
3     <section className="challenge">
4       <h2>{title}</h2>
5       <p className="challenge-time">
6         {targetTime} second {targetTime > 1 ? "s" : ""}
7       </p>
8       <button>Start</button>
9
10      <p className="">Time is running... / Timer inactive</p>
11    </section>
12  );
13 }
```

```
TimeStopper.jsx App.jsx x
src > App.jsx > App
5 function App() {
6   return (
7     <>
8       <Header />
9       <Player />
10      <div id="challenges">
11        <TimeStopper title="Lever 1" targetTime={1} />
12        <TimeStopper title="Lever 2" targetTime={5} />
13        <TimeStopper title="Lever 3" targetTime={10} />
14        <TimeStopper title="Lever 4" targetTime={15} />
15      </div>
16    </>
17  );
18 }
19
20 export default App;
```

LEVER 1

1 second

Start

Time is running... / Timer
inactive

LEVER 2

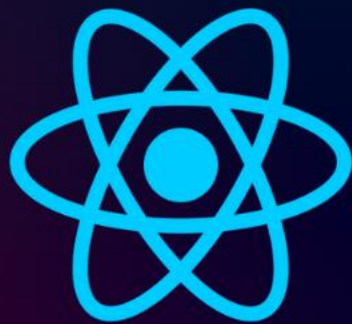
5 second s

Start

Time is running... / Timer
inactive

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.5

Refs & Portals

Thêm useState Timer



<http://Tuhoc.CC>





```
import {useState} from "react";

export default function TimeStopper({title, targetTime}) {
  const [timerStart, setTimerStart] = useState(false);
  const [timerExpired, setTimerExpired] = useState(false);

  function handleStart() {
    setTimeout(() => {
      setTimerExpired(true);
    }, targetTime * 1000);
    setTimerStart(true);
  }

  return (
    <section className="challenge">
      <h2>{title}</h2>
      {timerExpired && <p>Bạn đã thua</p>}
      <p className="challenge-time">
        {targetTime} second {targetTime > 1 ? "s" : ""}
      </p>
      <button onClick={handleStart}>{timerStart ? "Stop" : "Start"}</button>

      <p className={timerStart ? " active" : undefined}>{timerStart ? "Time is running" : "Timer inactive"}</p>
    </section>
  );
}
```

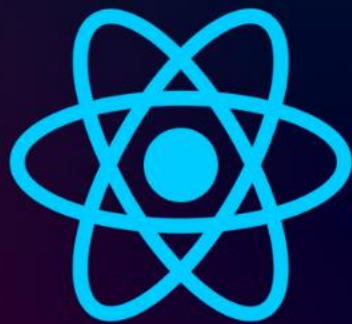
Hàm xử lý khi click vào nút

Nếu hết thời gian, hiện text bạn đã thua

Nếu đã nhấn bắt đầu, thì sẽ hiển thị text là Stop, nếu không hiển thị : Start

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.6

Refs & Portals

Sử dụng ref để xử lý Stop
Timer



<http://Tuhoc.CC>



6

Sử dụng ref để xử lý Stop Timer

✓ Hiện tại chúng ta chưa thể click để dừng



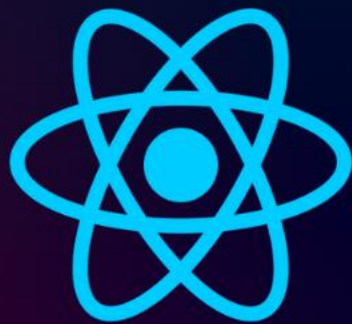
```
TimeStopper.jsx X
src > components > TimeStopper.jsx > TimeStopper
1  import {useState, useRef} from "react";
2
3  export default function TimeStopper({title, targetTime}) {
4    const timer = useRef();
5    const [timerStart, setTimerStart] = useState(false);
6    const [timerExpired, setTimerExpired] = useState(false);
7
8    function handleStart() {
9      timer.current = setTimeout(() => {
10        setTimerExpired(true);
11      }, targetTime * 1000);
12      setTimerStart(true);
13    }
14
15    function handleStop() {
16      clearTimeout(timer.current);
17      setTimerStart(false);
18    }
19  }
```

```
return (
  <section className="challenge">
    <h2>{title}</h2>
    {timerExpired && <p>Bạn đã thua</p>}
    <p className="challenge-time">
      {targetTime} second {targetTime > 1 ? "s" : ""}
    </p>
    <button onClick={timerStart ? handleStop : handleStart}>{timerStart ? "Stop" : "Start"}</button>

    <p className={timerStart ? " active" : undefined}>{timerStart ? "Time is running" : "Timer inactive"}</p>
  </section>
);
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.7

Refs & Portals

**Thêm Modal Popup - Kết
hợp useRef với dialog**



<http://Tuhoc.CC>



7

Thêm model popup

```
ResultModel.jsx x TimeStopper.jsx 1
src > components > ResultModel.jsx > ...
1 export default function ResultModel({result, targetTime, ref}) {
2   return (
3     <dialog ref={ref} className="result-modal">
4       <h2>You {result}</h2>
5       <p>
6         Thời gian đích: <strong>{targetTime}</strong> second</p>
7       <p>
8         Bạn đã dừng tại <strong>X</strong> second</p>
9       <form method="dialog">
10         <button>Close</button>
11       </form>
12     </dialog>
13   );
14 }
15
16
```

Google

dialog mdn

YOU LOSTThời gian đích: **1 second**Bạn đã dừng tại **X second**

Close

7

Thêm model popup

TimeStopper.jsx ✕

```
import ResultModel from "../ResultModel.jsx";
```

```
export default function TimeStopper({title, targetTime}) {
```

```
  const timer = useRef();
```

```
  const dialog = useRef();
```

Biến dialog, là 1 ref object, có 1 thuộc tính là current

```
  const [timerStart, setTimerStart] = useState(false);
```

```
  const [timerExpired, setTimerExpired] = useState(false);
```

```
  function handleStart() {
```

```
    timer.current = setTimeout(() => {
```

```
      setTimerExpired(true);
```

```
      dialog.current.showModal();
```

Hiện dialog khi hết thời gian

```
    }, targetTime * 1000);
```

```
    setTimerStart(true);
```

```
  }
```

```
  return (
```

```
    <>
```

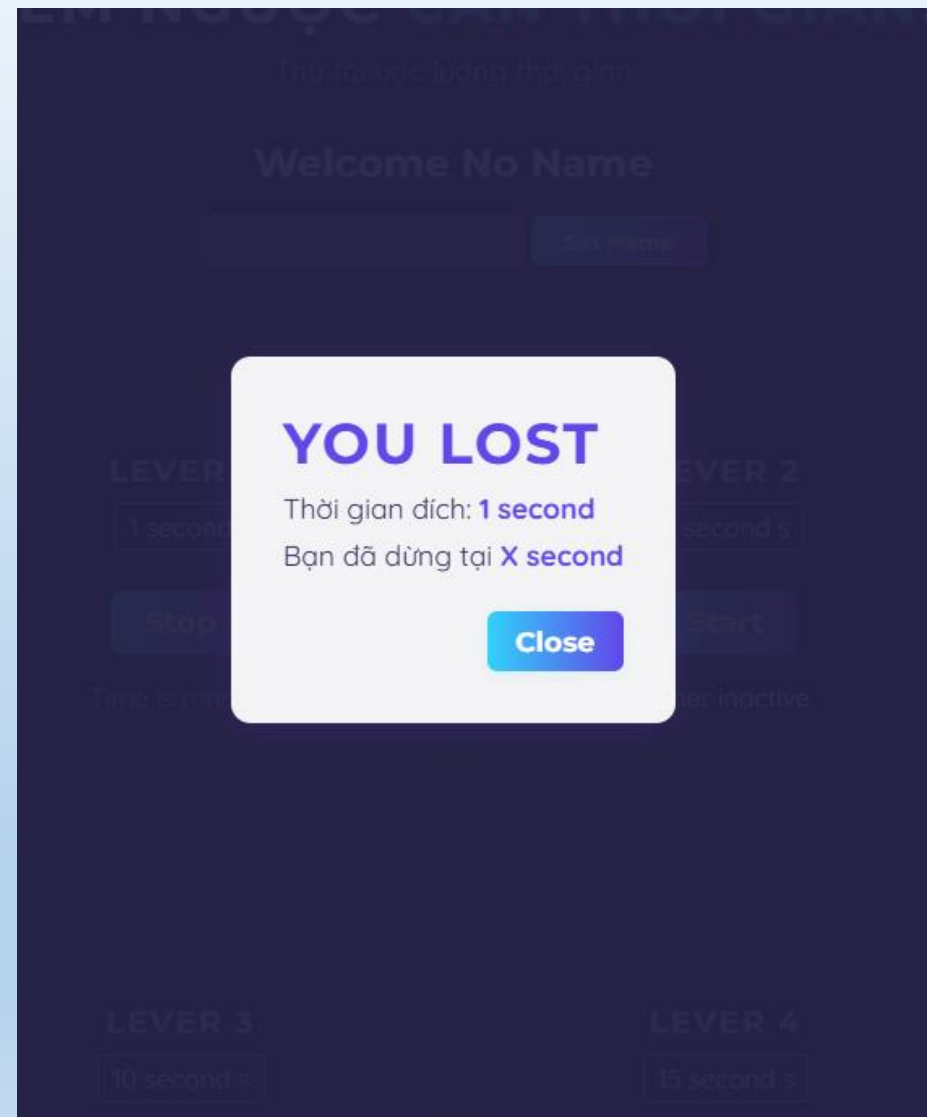
```
    <ResultModel ref={dialog} targetTime={targetTime} result="lost" />
```

```
    <section className="challenge">...
```

```
  </section>
```

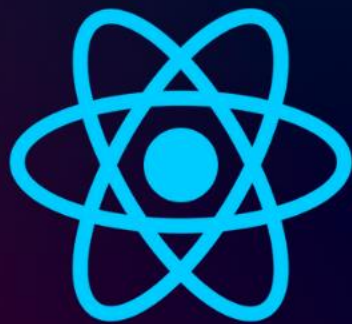
```
  </>
```

```
);
```



[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.8

Refs & Portals

useImperativeHandle Hook



<http://Tuhoc.CC>



8

useImperativeHandle Hook

- ✓ Vấn đề: Khi bạn làm dự án với Team, rất có thể người cùng team sẽ không có nhiều thời gian để đọc hiểu dự án
- ✓ Ví dụ vô tình họ đổi tên thẻ dialog của các bạn, thì ref gọi `dialog.showModal()` sẽ lỗi

TimeStopper.jsx 1

```
function handleStart() {  
  timer.current = setTimeout(() => {  
    setTimerExpired(true);  
    dialog.current.showModal();  
  }, targetTime * 1000);  
  setTimerStart(true);  
}
```

ResultModel.jsx X

```
export default function ResultModel({result, targetTime, ref}) {  
  return (  
    <dialog ref={ref} className="result-modal"> ...  
  </dialog>  
  );  
}
```



```
export default function ResultModel({result, targetTime, ref}) {  
  return (  
    <div ref={ref} className="result-modal"> ...  
  </div>  
  );  
}
```

✖ ▶ Uncaught TypeError: dialog.current.showModal is not a function
at [TimeStopper.jsx:14:22](#)

8

useImperativeHandle Hook

ResultModel.jsx ✕

```

1  import {useImperativeHandle, useRef} from "react";
2
3  export default function ResultModel({result, targetTime, ref}) {
4    const dialog = useRef();
5    useImperativeHandle(ref, () => {
6      return {
7        open() {
8          dialog.current.showModal();
9        },
10      };
11    });
12
13    return (
14 > <dialog ref={dialog} className="result-modal">...
15 </dialog>
16 );
17 }

```

TimeStopper.jsx 1 ✕

```

function handleStart() {
  timer.current = setTimeout(() => {
    setTimerExpired(true);
    // dialog.current.showModal();
    dialog.current.open();
  }, targetTime * 1000);
  setTimerStart(true);
}

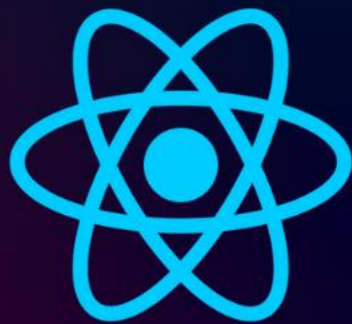
```

- ✓ *useImperativeHandle* cho phép component con chủ động chỉ định những hàm hoặc thuộc tính nào sẽ được "lộ" ra ngoài cho component cha truy cập qua ref.
- ✓ Component cha chỉ gọi được những hàm/thuộc tính được return trong object của *useImperativeHandle*, còn lại hoàn toàn không truy cập được.
- ✓ Tóm lại:

Chỉ những gì được khai báo trong *useImperativeHandle* mới dùng được từ cha.
 Các biến, hàm, DOM khác là private, cha không truy cập được.

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.9

Refs & Portals

Hiện modal
khi click stop



<http://Tuhoc.CC>



9

Hiện modal khi stop

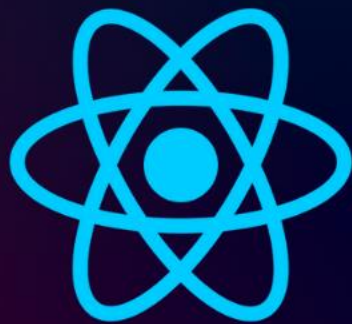
- ✓ Chúng ta cần tính toán thời điểm người chơi bấm dừng, còn cách đích đến bao lâu? → Suy ra điểm số
- ✓ *Settimeout* sẽ không cho ta thông tin này, chúng ta sử dụng *setInterval()*

```
TimeStopper.jsx X
src > components > TimeStopper.jsx > TimeStopper
4 export default function TimeStopper({title, targetTime}) {
5   const timer = useRef();
6   const dialog = useRef();
7
8   const [timeRemaining, setTimeRemaining] = useState(targetTime * 1000);
9   const timerIsActive = timeRemaining > 0 && timeRemaining < targetTime * 1000;
10  if (timeRemaining <= 0) {
11    clearInterval(timer.current);
12    dialog.current.open();
13  }
14  function handleStart() {
15    timer.current = setInterval(() => {
16      setTimeRemaining((prevTimeRemaining) => prevTimeRemaining - 10);
17    }, 10);
18  }
19
20  function handleStop() {
21    dialog.current.open();
22    clearInterval(timer.current);
23  }
24
```

```
return (  
  <>  
    <ResultModel ref={dialog} targetTime={targetTime} result="lost" />  
    <section className="challenge">  
      <h2>{title}</h2>  
      <p className="challenge-time">  
        {targetTime} second {targetTime > 1 ? "s" : ""}  
      </p>  
      <button onClick={timerIsActive ? handleStop : handleStart}>{timerIsActive ? "Stop" : "Start"}</button>  
  
      <p className={timerIsActive ? " active" : undefined}>{timerIsActive ? "Time is running" : "Timer inactive"}</p>  
    </section>  
  </>  
) ;
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.10

Refs & Portals

Xử lý thông báo thắng - thua



<http://Tuhoc.CC>



- ✓ Chúng ta đang gán cố định thông báo là lost

```
TimeStopper.jsx
return (
  <>
    <ResultModel ref={dialog} targetTime={targetTime} result="lost" />
    <section className="challenge">
      <h2>{title}</h2>
      <p className="challenge-time">
        {targetTime} second {targetTime > 1 ? "s" : ""}
      </p>
      <button onClick={timerIsActive ? handleStop : handleStart}>{timerIsActive ? "Stop" : "Start"}</button>
      <p className={timerIsActive ? " active" : undefined}>{timerIsActive ? "Time is running" : "Time is not running"}</p>
    </section>
  </>
);
```

- ✓ Chúng ta sẽ chuyển remainingTime vào để tính toán và đưa ra thông báo thắng hay thua, và điểm số sau này

```
function handleReset() {
  setTimeRemaining(targetTime * 1000);
}
```

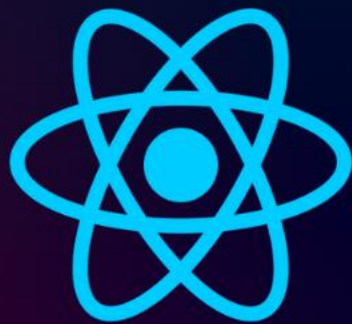
```
<ResultModel ref={dialog} targetTime={targetTime} remainingTime={timeRemaining} onReset={handleReset} />
```


✓ Chúng ta cần thêm prop vào *ResultModel*

```
TimeStopper.jsx ResultModel.jsx X
src > components > ResultModel.jsx > ResultModel
1 import {useImperativeHandle, useRef} from "react";
2
3 export default function ResultModel({targetTime, ref, remainingTime, onReset}) {
4   const dialog = useRef();
5   const userLost = remainingTime <= 0;
6   const formattedRemainingTime = (remainingTime / 1000).toFixed(2);
7
8   useImperativeHandle(ref, () => { ...
14   });
15
16   return (
17     <dialog ref={dialog} className="result-modal">
18       /* <h2>You {result}</h2> */
19       {userLost && <h2>You Lost</h2>}
20       <p>
21         Thời gian đích: <strong>{targetTime}</strong> second</strong>
22       </p>
23       <p>
24         Bạn đã dừng tại <strong>{formattedRemainingTime}</strong> second</strong>
25       </p>
26       <form method="dialog" onSubmit={onReset}>
27         <button>Close</button>
28       </form>
29     </dialog>
30   );
31 }
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.11

Refs & Portals

Hiển thị kết quả
Tính điểm khi dừng



<http://Tuhoc.CC>



ResultModel.jsx 1

```
export default function ResultModel({targetTime, ref, remainingTime, onReset}) {  
  const dialog = useRef();  
  const userLost = remainingTime <= 0;  
  const formattedRemainingTime = (remainingTime / 1000).toFixed(2);  
  const score = Math.round((1 - remainingTime / (targetTime * 1000)) * 100);  
}
```

- ✓ **targetTime * 1000**: Tổng thời gian mục tiêu (miligiây).
- ✓ **remainingTime**: Thời gian còn lại khi người chơi dừng (miligiây).
- ✓ **remainingTime / (targetTime * 1000)**: Tỷ lệ phần trăm thời gian còn lại so với tổng thời gian.
- ✓ **1 - ...**: Tỷ lệ phần trăm thời gian đã sử dụng so với tổng thời gian.
- ✓ **(1 - ...) * 100**: Chuyển sang phần trăm (vd: 0.85 → 85%).
- ✓ **Math.round(...)**: Làm tròn số điểm.

targetTime = 10 s, dừng khi đã chạy được 8s

remainingTime = 2000;

score = Math.round((1 - 2000/10000) * 100) => (1 - 0.2) * 100 = 80

 ResultModel.jsx ✕

```
return (  
  <dialog ref={dialog} className="result-modal">  
    { /* <h2>You {result}</h2> */ }  
    {userLost && <h2>You Lost</h2>}  
    {!userLost && <h2>Điểm của bạn: {score}</h2>}  
    <p>  
      Thời gian đích: <strong>{targetTime} second</strong>  
    </p>  
    <p>  
      Bạn đã dừng tại <strong>{formattedRemainingTime} second</strong>  
    </p>  
    <form method="dialog" onSubmit={onReset}>  
      <button>Close</button>  
    </form>  
  </dialog>  
)
```

ĐIỂM CỦA BẠN: 99

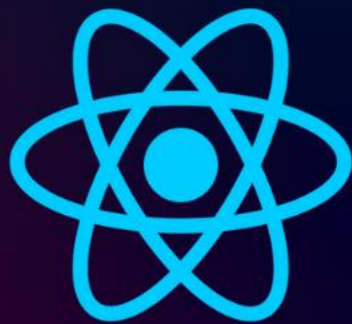
Thời gian đích: 5 second

Bạn đã dừng tại 0.03 second

Close

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 18.12

Refs & Portals

React Portals



<http://Tuhoc.CC>



- ✓ *Portals là một tính năng trong React cho phép bạn render một component, gắn thẳng vào vị trí bất kỳ trên DOM (ví dụ: ngoài cùng trong <body>)*

```

<body>
  <div id="modal"></div>
  <div id="content">
    <div id="root">
      <header>...</header>
      <section id="player">...</section>
      <div id="challenges">flex == $0
        <dialog class="result-modal">...</dialog>
        <section class="challenge">...</section> flex
        <dialog class="result-modal" open>...</dialog>
        scroll top-layer (1)
        <section class="challenge">...</section> flex
        <dialog class="result-modal">...</dialog>
        <section class="challenge">...</section> flex
        <dialog class="result-modal">...</dialog>
        <section class="challenge">...</section> flex
      </div>
    </div>
  </div>
</body>

```



```

<body>
  <div id="modal">
    <dialog class="result-modal">...</dialog>
    <dialog class="result-modal" open>...</dialog>
    <dialog class="result-modal">...</dialog> == $0
    <dialog class="result-modal">...</dialog>
  </div>
  <div id="content">...</div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>

```

12

Portals trong React

 ResultModel.jsx ✕

```
return (  
  <dialog ref={dialog} className="result-modal"> ...  
  </dialog>  
);
```



```
import {createPortal} from "react-dom";
```

```
return createPortal(  
  <dialog ref={dialog} className="result-modal"> ...  
  </dialog>,  
  document.getElementById("modal")  
);
```

```
import { createPortal } from 'react-dom';
```

```
// JSX
```

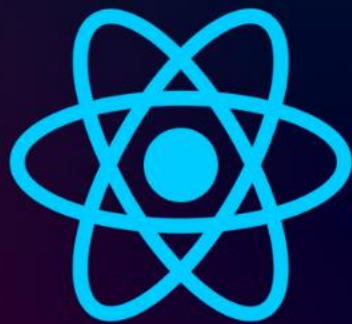
```
return createPortal(  
  <mã JSX cần render />,  
  document.getElementById('modal')  
  // vị trí muốn render trong DOM  
);
```

 index.html

```
<body>  
  <div id="modal"></div>  
  <div id="content">  
    <div id="root"></div>  
  </div>  
  <script type="module" src="/src/main.jsx"></script>  
</body>
```


[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 19

useReducer ()

Quản lý state phức tạp



<http://Tuhoc.CC>



1

Ví dụ ôn tập useState

```
App.jsx LoginState.jsx X
src > assets > components > LoginState.jsx > ...
1 import {useState} from "react";
2
3 function LoginState() {
4   const [isLoggedIn, setIsLoggedIn] = useState(false);
5   function handleLogout() {
6     setIsLoggedIn(false);
7   }
8
9   function handleLogin() {
10    setIsLoggedIn(true);
11  }
12
13  return (
14    <div>
15      <h2>{isLoggedIn ? "Chào mừng xxx" : "Bạn chưa đăng nhập"}</h2>
16      <button onClick={handleLogin}>Đăng nhập</button>
17      <button onClick={handleLogout}>Đăng xuất</button>
18    </div>
19  );
20 }
21
22 export default LoginState;
```

```
App.jsx LoginState.jsx X
src > assets > components > LoginState.jsx > ...
1 import {useState} from "react";
2
3 function LoginState() {
4   const [isLoggedIn, setIsLoggedIn] = useState(false);
5
6   return (
7     <div>
8       <h2>{isLoggedIn ? "Chào mừng xxx" : "Bạn chưa đăng nhập"}</h2>
9       <button onClick={()=>setIsLoggedIn(true)}>Đăng nhập</button>
10      <button onClick={()=>setIsLoggedIn(false)}>Đăng xuất</button>
11    </div>
12  );
13 }
14
15 export default LoginState;
16
```

Chào mừng xxx

Đăng nhập

Đăng xuất

```
function App() {
  return (
    <>
      <LoginState />
    </>
  );
}
```

2

Ví dụ sử dụng useReducer

```

//Step 1: import useReducer
import {useReducer} from "react";

//step 2: Khởi tạo giá trị ban đầu
const initState = false;

//Step 3: khai báo action
const LOGIN = "login";
const LOGOUT = "logout";

//Step 4: khai báo function reducer ( sẽ làm gì với từng action)
function reducerLogin(state, action) {
  switch (action) {
    case LOGIN:
      return true;
    case LOGOUT:
      return false;
    default:
      throw new Error("Action không hợp lệ!");
  }
}

```

```

function LoginUseReducer() {
  const [isLoggedIn, dispatch] = useReducer(reducerLogin, initState);
  // Step 5: [giá trị hiện tại, dispatch: hàm gửi action lên reducer]
  // = (hàm reducer, lọc và xử lý action gửi lên, Giá trị khởi tạo ban đầu)

  return (
    <div>
      <h2>{isLoggedIn ? "Bạn đã đăng nhập" : "Bạn chưa đăng nhập"}</h2>
      { /* Step 6: Dùng dispatch để gửi action */ }
      <button onClick={() => dispatch(LOGIN)}>Đăng nhập</button>
      <button onClick={() => dispatch(LOGOUT)}>Đăng xuất</button>
    </div>
  );
}

export default LoginUseReducer;

```

✓ **Reducer là một hàm nhận vào 2 tham số:**

state: giá trị trạng thái hiện tại

action: hành động (thông điệp) gửi lên để yêu cầu thay đổi state

✓ **Nhiệm vụ của reducer:**

Xử lý action: Dựa vào action gửi lên, xác định cần cập nhật lại state như thế nào.

Trả về state mới: Dựa vào action, reducer sẽ trả về giá trị state mới.

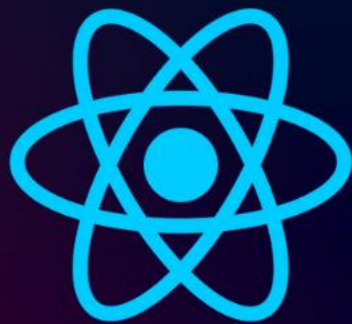
3

useState hay useReducer ?

No	Trường hợp sử dụng	useState	useReducer
1	Đơn giản, ít logic	✓	Có thể dùng nhưng hơi dư thừa
2	State phức tạp (object, array nhiều trường)	✗	✓
3	Dễ mở rộng, bảo trì	✗	Rất tốt
4	Học lên Redux dễ		Rất thuận lợi

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 20.1

useContext ()

**Hạn chế của useState với
cách truyền props drilling**



<http://Tuhoc.CC>



1

Nhược điểm của useState()

- ✓ *useState bị bất tiện khi phải truyền dữ liệu qua nhiều tầng component (props drilling)*
- ✓ *Có những component chỉ đóng vai trò trung gian, không dùng đến props nhưng vẫn phải truyền props qua cho component con, gây thừa thãi và rối code.*
- ✓ *Nếu sau này thay đổi logic, bỏ hoặc thay đổi component trung gian, mà không cập nhật lại luồng props, sẽ dễ bị lỗi, và rất khó kiểm soát.*

```
App (hiển thị tổng số sản phẩm trong giỏ)
├── ProductList
│   └── ProductItem
│       └── CartButton (nút thêm sản phẩm)
```



1

Nhược điểm của useState()

App.jsx

```
1 import MainShop from "../components/Shop/MainShop.jsx";
2
3 function App() {
4   return (
5     <>
6       <MainShop />
7     </>
8   );
9 }
10
11 export default App;
```

App (hiển thị tổng số sản phẩm trong giỏ)

└─ ProductList

└─ ProductItem

└─ CartButton (nút thêm sản phẩm)

App.jsx

MainShop.jsx

ProductList.jsx

ProductItem.jsx

Cart

```
src > components > Shop > MainShop.jsx > ...
1 import {useState} from "react";
2 import ProductList from "../ProductList";
3
4 function MainShop() {
5   const [cartCount, setCartCount] = useState(0);
6
7   function handleCart() {
8     setCartCount((prev) => prev + 1);
9   }
10
11   return (
12     <div>
13       <h1>Giỏ hàng: {cartCount} sản phẩm</h1>
14       <ProductList addToCart={handleCart} />
15     </div>
16   );
17 }
18
19 export default MainShop;
```

1

Nhược điểm của useState()

App (hiển thị tổng số sản phẩm trong giỏ)

└─ ProductList

└─ ProductItem

└─ CartButton (nút thêm sản phẩm)

```
App.jsx MainShop.jsx ProductList.jsx X ProductItem.jsx C
src > components > Shop > ProductList.jsx > ...
1 import ProductItem from "../ProductItem";
2
3 function ProductList({addToCart}) {
4   return (
5     <div>
6       <h2>Danh sách sản phẩm</h2>
7       <ProductItem addToCart={addToCart} />
8     </div>
9   );
10 }
11
12 export default ProductList;
13
```

```
App.jsx MainShop.jsx ProductList.jsx X ProductItem.jsx X
src > components > Shop > ProductItem.jsx > ...
1 import CartButton from "../CartButton";
2
3 function ProductItem({addToCart}) {
4   return (
5     <div>
6       <h3>Sản phẩm A</h3>
7       <CartButton addToCart={addToCart} />
8     </div>
9   );
10 }
11
12 export default ProductItem;
```

1

Nhược điểm của useState()

App (hiển thị tổng số sản phẩm trong giỏ)

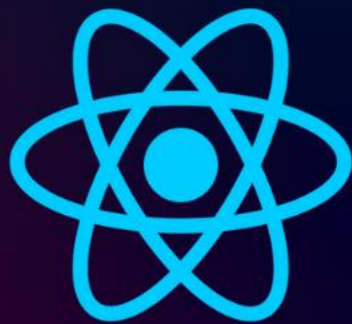
- └─ ProductList
 - └─ ProductItem
 - └─ CartButton (nút thêm sản phẩm)

```
App.jsx MainShop.jsx ProductList.jsx ProductItem.jsx X
src > components > Shop > ProductItem.jsx > ...
1 import CartButton from "../CartButton";
2
3 function ProductItem({addToCart}) {
4   return (
5     <div>
6       <h3>Sản phẩm A</h3>
7       <CartButton addToCart={addToCart} />
8     </div>
9   );
10 }
11
12 export default ProductItem;
```

```
App.jsx MainShop.jsx ProductList.jsx ProductItem.jsx CartButton.jsx X
src > components > Shop > CartButton.jsx > ...
1 function CartButton({addToCart}) {
2   return <button onClick={addToCart}>Thêm vào giỏ hàng</button>;
3 }
4
5 export default CartButton;
6
```


[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 20.2

useContext ()

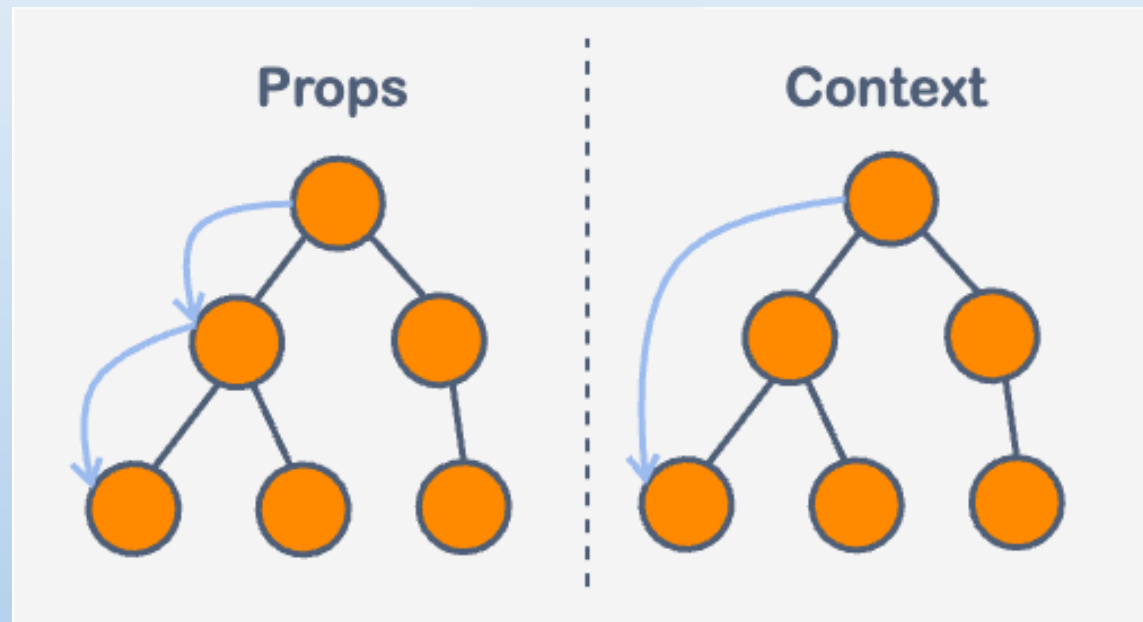
Truyền dữ liệu nhiều tầng



<http://Tuhoc.CC>



- ✓ *useContext* là một hook trong React dùng để *truyền dữ liệu giữa các component* mà *không cần phải truyền props qua từng tầng (props drilling)*.



3

Các bước sử dụng useContext()

✓ Bước 1: Tạo context

```
import { createContext } from "react";  
const CartContext = createContext();  
export default CartContext;
```

✓ Bước 2: Dùng Provider để “bọc” cây component

```
<CartContext.Provider value={{ cartCount, handleCart }}>  
  {/* các component con */}  
</CartContext.Provider>
```

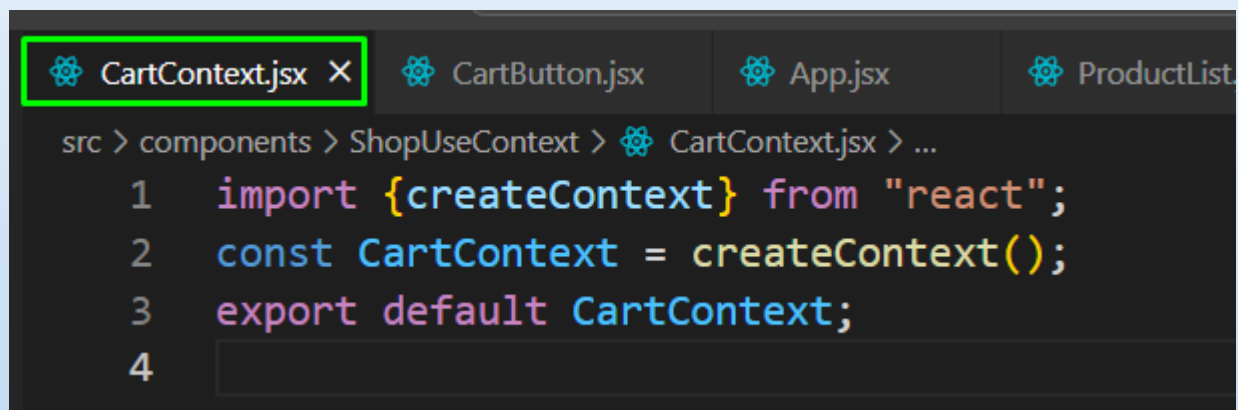
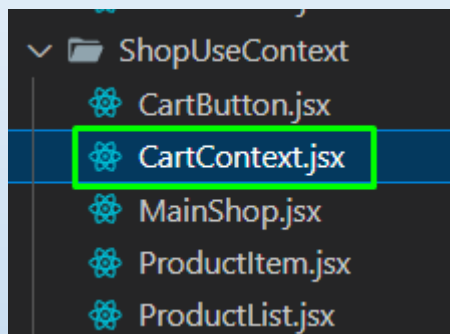
✓ Bước 3: Dùng useContext ở bất kỳ component con nào để “lấy” dữ liệu ra

```
import { useContext } from "react";  
import CartContext from "../CartContext";  
  
const { cartCount, handleCart } = useContext(CartContext);
```

2

Các bước sử dụng useContext()

✓ Bước 1: Tạo file context



2

Các bước sử dụng useContext()

✓ Bước 2: Dùng Provider để “bọc” cây component

MainShop.jsx ✕

```
return (  
  <div>  
    <h1>Giỏ hàng: {cartCount} sản phẩm</h1>  
    <ProductList addToCart={handleCart} />  
  </div>  
);
```

Bỏ hết props trung gian

Truyền giá trị muốn chia sẻ (state & hàm) vào prop value của Provider

Tất cả component bên trong đều truy cập được cartCount, handleCart qua useContext

MainShop.jsx ✕

```
src > components > ShopUseContext > MainShop.jsx > MainShop  
1  import {useState} from "react";  
2  import ProductList from "../ProductList";  
3  import CartContext from "../CartContext.jsx";  
4  
5  function MainShop() {  
6    const [cartCount, setCartCount] = useState(0);  
7  
8    function handleCart() {  
9      setCartCount((prev) => prev + 1);  
10   }  
11  
12   return (  
13     <CartContext.Provider value={{cartCount, handleCart}}>  
14       <div>  
15         <h1>Giỏ hàng: {cartCount} sản phẩm</h1>  
16         <ProductList />  
17       </div>  
18     </CartContext.Provider>  
19   );  
20 }  
21  
22 export default MainShop;
```

2

Các bước sử dụng useContext()

✓ **Bước 3:** Dùng useContext ở bất kỳ component con nào để “lấy” dữ liệu ra

 CartButton.jsx ✕

```
function CartButton({addToCart}) {  
  return <button onClick={addToCart}>Thêm vào giỏ hàng</button>;  
}  
  
export default CartButton;
```

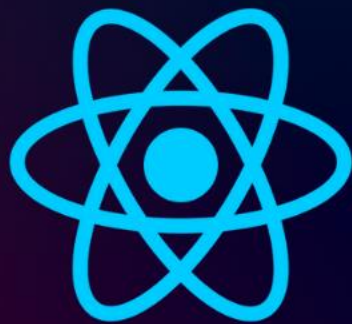
```
import {useContext} from "react";  
import CartContext from "../CartContext.jsx";
```



```
function CartButton() {  
  const {cartCount, handleCart} = useContext(CartContext);  
  return (  
    <>  
      <button onClick={handleCart}>Thêm vào giỏ hàng</button>  
      <p>Số item trong giỏ hiện tại : {cartCount}</p>  
    </>  
  );  
}  
  
export default CartButton;
```

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 21.1

useEffect ()

React useEffect Là Gì



<http://Tuhoc.CC>



1

useEffect ()

- ✓ **useEffect** trong React là một hook dùng để xử lý các “**side effect**” (tác động phụ) trong component, ví dụ như: **gọi API, cập nhật DOM, setTimeout, setInterval, hoặc lắng nghe/hủy lắng nghe event...**
- ✓ **Hiểu đơn giản, sau khi** React đã render xong component. **useEffect = "React ơi, sau khi render xong, hãy giúp tôi làm cái việc này nhé!"** (Ví dụ: gọi API - **Application Programming Interface** , **setTimeout, add event listener,...**)
- ✓ **Cách dùng cơ bản:**

```
import {useEffect} from "react";
```

```
useEffect(CallBack, [deps])
```

```
useEffect(() => {  
  // code xử lý side effect ở đây  
}, [dependency]);
```

1. Nếu **không** truyền dependency, hàm **callback** chạy sau mỗi lần render.
2. Nếu mảng dependency là [] (rỗng), hàm chỉ chạy 1 lần sau khi component mount.
3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

1

useEffect () không truyền dependency

1. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.
2. Nếu mảng dependency là [] (rỗng), hàm chỉ chạy 1 lần sau khi component mount.
3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

```
App.jsx  main.jsx  DemoNoDeps.jsx X
src > components > DemoNoDeps.jsx > ...
1  import {useEffect, useState} from "react";
2
3  export default function DemoNoDeps() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      console.log("useEffect chạy sau mỗi lần render");
8    });
9
10   return (
11     <div>
12       <h3>Không truyền dependency</h3>
13       <button onClick={() => setCount(count + 1)}>Tăng</button>
14       <p>Giá trị: {count}</p>
15       {console.log("Render lại giao diện")}
16     </div>
17   );
18 }
```

Không truyền dependency

Tăng

Giá trị: 0

Render lại giao diện

useEffect chạy sau mỗi lần render

Rõ ràng: UseEffect chạy sau khi giao diện đã render xong

Tăng

Giá trị: 2

Component rerender thì useEffect cũng được gọi

Render lại giao diện

useEffect chạy sau mỗi lần render

Render lại giao diện

useEffect chạy sau mỗi lần render

Render lại giao diện

useEffect chạy sau mỗi lần render

2

useEffect () dependency là []

1. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.
2. Nếu mảng dependency là [] (rỗng), hàm chỉ chạy 1 lần sau khi component mount.
3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

```
App.jsx  main.jsx  DemoNoDeps.jsx  DemoEmptyDeps.jsx X
> components > DemoEmptyDeps.jsx > ...
1  import {useEffect, useState} from "react";
2
3  export default function DemoEmptyDeps() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      console.log("useEffect chỉ chạy 1 lần khi component mount");
8    }, []);
9
10   return (
11     <div>
12       <h3>Dependency là []</h3>
13       <button onClick={() => setCount(count + 1)}>Tăng</button>
14       <p>Giá trị: {count}</p>
15       {console.log("Render lại giao diện")}
16     </div>
17   );
18 }
```

** UseEffect chạy sau khi giao diện đã render xong*

Dependency là []

Tăng

Giá trị: 3

Render lại giao diện

useEffect chỉ chạy 1 lần khi component mount

3 Render lại giao diện

** UseEffect chỉ chạy 1 lần ĐUY NHẤT khi component được mount*

3

useEffect () với dependency

1. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.
2. Nếu mảng dependency là [] (rỗng), hàm chỉ chạy 1 lần sau khi component mount.
3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

Giá trị dependency trong useEffect là một mảng – bạn có thể truyền vào nhiều biến (state, props...) mà bạn muốn theo dõi. Chỉ cần 1 trong các giá trị trong mảng này thay đổi, hàm callback trong useEffect sẽ được gọi lại.

```
import {useEffect, useState} from "react";

export default function DemoWithDeps() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("useEffect chạy khi count đổi:", count);
  }, [count]);

  return (
    <div>
      <h3>Dependency là [count]</h3>
      <button onClick={() => setCount(count + 1)}>Tăng</button>
      <p>Giá trị: {count}</p>
      {console.log("Render lại giao diện")}
    </div>
  );
}
```

** useEffect chạy sau khi giao diện đã render xong*

Dependency là [count]

Tăng

Giá trị: 2

Render lại giao diện

useEffect chạy khi count đổi: 0

Render lại giao diện

useEffect chạy khi count đổi: 1

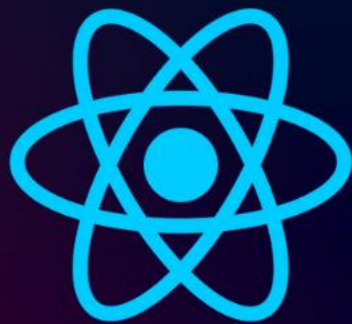
Render lại giao diện

useEffect chạy khi count đổi: 2

** Count được đưa vào theo dõi, nếu nó thay đổi, thì useEffect sẽ được gọi*

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 21.2

`useEffect ()`

call API



<http://Tuhoc.CC>



4

useEffect () ví dụ : Call API

1. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.
2. Nếu mảng dependency là [] (rỗng), hàm chỉ chạy 1 lần sau khi component mount.
3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

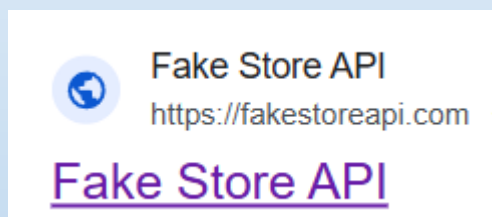
JSON – dạng dữ liệu phổ biến khi làm việc với API

→ Thường dùng để truyền dữ liệu giữa frontend và backend.

** UseEffect chạy sau khi giao diện đã render xong*



<https://fakestoreapi.com/products>



```
{
  "id": 1,
  "title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",
  "price": 109.95,
  "description": "Your perfect pack for everyday use and walks in the f",
  "category": "men's clothing",
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "rating": {
    "rate": 3.9,
    "count": 120
  }
},
```

Đây là một mảng JSON.

Trong mảng này, mỗi phần tử là một đối tượng sản phẩm.

Mỗi đối tượng sản phẩm lại có các thuộc tính như tên, giá, mô tả, hình ảnh...

Thậm chí có những thuộc tính là một đối tượng con, ví dụ như rating (gồm số sao và số lượt đánh giá).

Cấu trúc này rất phổ biến khi frontend nhận dữ liệu từ backend (API trả về).

4

useEffect () ví dụ : Call API

1. Nếu không sử dụng useEffect

```
App.jsx DemoNoDeps.jsx X
src > components > DemoNoDeps.jsx > ...
1 import {useState} from "react";
2
3 export default function DemoNoDeps() {
4   const [count, setCount] = useState(0);
5
6   {console.log("Bắt đầu đến fetch")}
7   fetch("https://fakestoreapi.com/products/1")
8     .then((res) => res.json())
9     .then((json) => console.log(json));
10  {console.log("Chương trình đã chạy qua fetch")}
11
12  return (
13    <div>
14      <h3>Không truyền dependency</h3>
15      <button onClick={() => setCount(count + 1)}>Tăng</button>
16      <p>Giá trị: {count}</p>
17      {console.log("Render lại giao diện")}
18    </div>
19  );
20 }
```

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

Render lại giao diện

{id: 1, title: 'Fjallraven - Fo
and walks in th...to 15 inches) i

fetch là hàm bất đồng bộ (asynchronous). Khi bạn gọi fetch(), nó không chờ kết quả mà tiếp tục thực thi các dòng code phía sau ngay lập tức.

Chương trình chạy tuần tự, theo đúng luồng từ trên xuống dưới, bất kỳ khi nào Component được re-render

4

useEffect () ví dụ : Call API

1. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.

2. Nếu không truyền dependency, hàm **callback** chạy sau mỗi lần render.

```
App.jsx DemoNoDeps.jsx X
src > components > DemoNoDeps.jsx > ...
1 import {useEffect, useState} from "react";
2
3 export default function DemoNoDeps() {
4   const [count, setCount] = useState(0);
5
6   useEffect(()=>{
7     {console.log("Bắt đầu đến fetch")}
8     fetch("https://fakestoreapi.com/products/1")
9       .then((res) => res.json())
10      .then((json) => console.log(json));
11     {console.log("Chương trình đã chạy qua fetch")}
12   }
13 )
14
15 return (
16   <div>
17     <h3>Không truyền dependency</h3>
18     <button onClick={() => setCount(count + 1)}>Tăng</button>
19     <p>Giá trị: {count}</p>
20     {console.log("Render lại giao diện")}
21   </div>
22 );
23 }
```

Render lại giao diện

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Foldse
and walks in th...to 15 inches) in th
```

Render lại giao diện

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Foldse
and walks in th...to 15 inches) in t
```

Render lại giao diện

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Foldse
and walks in th...to 15 inches) in t
```

Khi cập nhật state đếm số, component lại fetch lại API → Rõ ràng đây là việc không cần thiết, phát sinh thêm request không cần thiết đến máy chủ

4

useEffect () ví dụ : Call API

2. Nếu mảng **dependency** là `[]` (rỗng), hàm chỉ chạy 1 lần sau khi **component mount**.

3. Nếu mảng **dependency** là `[]` (rỗng), hàm chỉ chạy 1 lần sau khi **component mount**.

```

App.jsx DemoNoDeps.jsx DemoEmptyDeps.jsx X
src > components > DemoEmptyDeps.jsx > DemoEmptyDeps
1  import {useEffect, useState} from "react";
2
3  export default function DemoEmptyDeps() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      {console.log("Bắt đầu đến fetch")}
8      fetch("https://fakestoreapi.com/products/1")
9        .then((res) => res.json())
10       .then((json) => console.log(json));
11      {console.log("Chương trình đã chạy qua fetch")}
12    }, []);
13
14
15
16    return (
17      <div>
18        <h3>Dependency là []</h3>
19        <button onClick={() => setCount(count + 1)}>Tăng</button>
20        <p>Giá trị: {count}</p>
21        {console.log("Render lại giao diện")}
22      </div>
23    );
24  }

```

Render lại giao diện

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Folsom
and walks in th...to 15 inches) in
```

3 Render lại giao diện

Hàm callback trong **useEffect** Chỉ thực thi 1 lần duy nhất

4

useEffect () ví dụ : Call API

3. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

4. Chạy lại mỗi khi giá trị trong mảng dependency thay đổi

Giá trị dependency trong useEffect là một mảng – bạn có thể truyền vào nhiều biến (state, props...) mà bạn muốn theo dõi. Chỉ cần 1 trong các giá trị trong mảng này thay đổi, hàm callback trong useEffect sẽ được gọi lại.

```

App.jsx DemoWithDeps.jsx X DemoNoDeps.jsx DemoEmptyDeps.jsx
src > components > DemoWithDeps.jsx > DemoWithDeps
1 import {useEffect, useState} from "react";
2
3 export default function DemoWithDeps() {
4   const [count, setCount] = useState(0);
5
6   useEffect(() => {
7     console.log("Bắt đầu đến fetch");
8     fetch("https://fakestoreapi.com/products/1")
9       .then((res) => res.json())
10      .then((json) => console.log(json));
11     console.log("Chương trình đã chạy qua fetch");
12   }, [count]);
13
14   return (
15     <div>
16       <h3>Dependency là [count]</h3>
17       <button onClick={() => setCount(count + 1)}>Tăng</button>
18       <p>Giá trị: {count}</p>
19       {console.log("Render lại giao diện")}
20     </div>
21   );
22 }

```

Render lại giao diện

Bắt đầu đến fetch

Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Foldsack No
and walks in th...to 15 inches) in the pad
```

Render lại giao diện

Bắt đầu đến fetch

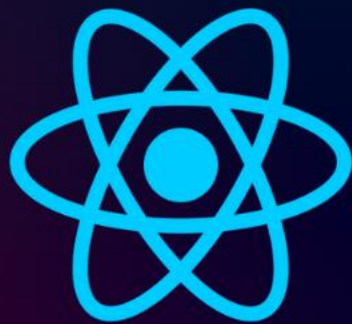
Chương trình đã chạy qua fetch

```
{id: 1, title: 'Fjallraven - Foldsack No
and walks in th...to 15 inches) in the pad
```

Hàm callback trong useEffect thực thi khi count thay đổi (vì count đang được theo dõi)

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 21.3

useEffect ()

Cleanup
setInterval/setTimeout



<http://Tuhoc.CC>



5

useEffect () : Làm việc với setInterval

```
TimeDemo.jsx Parent.jsx X App.jsx main.jsx
src > components > Parent.jsx > ...
1  import {useState} from "react";
2  import TimeDemo from "../TimeDemo.jsx";
3
4  function Parent() {
5    const [show, setShow] = useState(true);
6
7    return (
8      <div>
9        <button onClick={() => setShow((s) => !s)}>{show ? "Ẩn Component" : "Hiện Component"}</button>
10       {show && <TimeDemo />}
11     </div>
12   );
13 }
14 export default Parent;
```

- ✓ Mount - component được gắn vào DOM.
- ✓ Unmount - component bị gỡ khỏi DOM.

Ẩn Component

Unmount component để thử !

Count: 0

5

useEffect () : Làm việc với setInterval

```

TimeDemo.jsx x
src > components > TimeDemo.jsx > TimeDemo
1  import {useEffect, useState} from "react";
2
3  export default function TimeDemo() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      const intervalId = setInterval(() => {
8        console.log("set interval được gọi");
9        setCount((prev) => prev + 1); // Gây update state
10     }, 1000); // Sau 1 giây
11
12     //cleanup !
13     return () => {
14       clearInterval(intervalId);
15       console.log("đã clear time interval");
16     };
17   }, []);
18
19   return (
20     <div>
21       <h3>Unmount component để thử !</h3>
22       <p>Count: {count}</p>
23     </div>
24   );
25 }

```

Khi unmount component, setInterval sẽ tiếp tục chạy kể cả khi component bị gỡ khỏi DOM → gây memory leak.

112 set interval được gọi

>

*Clean up chỉ đơn giản là 1 hàm, khai báo trong return, bên trong useEffect(). React sẽ gọi hàm này khi **component bị unmount** - hoặc khi **effect được chạy lại với dependency mới**, đảm bảo clear được Interval tránh rò rỉ bộ nhớ*

3 set interval được gọi
đã clear time interval

3 set interval được gọi
đã clear time interval

6

useEffect () : Làm việc với setTimeout

```

TimeDemo.jsx x
src > components > TimeDemo.jsx > ...
1  import {useEffect, useState} from "react";
2
3  export default function TimeDemo() {
4    const [count, setCount] = useState(0);
5    useEffect(() => {
6      const setTimeoutId = setTimeout(() => {
7        console.log("set timeout được gọi");
8        setCount((prev) => prev + 1);
9      }, 1000);
10     //cleanup
11     return () => {
12       clearTimeout(setTimeoutId);
13       console.log("Đã clear timeout");
14     };
15   }, [count]);
16   return (
17     <>
18       <h3>Mount/ Unmount component để thử</h3>
19       <p>Count: {count}</p>
20     </>
21   );
22 }

```

*Clean up chỉ đơn giản là 1 hàm, khai báo trong return, bên trong useEffect(). React sẽ gọi hàm này khi **component bị unmount** - hoặc khi effect được chạy lại với dependency mới, đảm bảo clear được Interval tránh rò rỉ bộ nhớ*

Ẩn Component

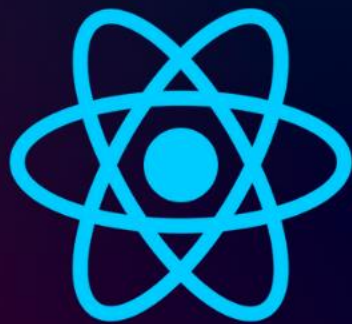
Unmount component để thử !

Count: 2

set out được gọi
đã clear time out
set out được gọi
đã clear time out

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 21.4

`useEffect ( )`

`useEffect` với event



<http://Tuhoc.CC>



7

useEffect () : Làm việc với Event



dom event w3

*Clean up chỉ đơn giản là 1 hàm, khai báo trong return, bên trong useEffect(). React sẽ gọi hàm này khi **component bị unmount** - hoặc khi effect được chạy lại với dependency mới, đảm bảo clear được Interval tránh rò rỉ bộ nhớ*

```
src > components > Parent.jsx > [default]
1  import {useState} from "react";
2  import ResizeEventDemo from "../ResizeEventDemo.jsx";
3
4  function Parent() {
5    const [show, setShow] = useState(true);
6
7    return (
8      <div>
9        <button onClick={() => setShow((s) => !s)}>{show ? "Ẩn Component" : "Hiện Component"}</button>
10       {show && <ResizeEventDemo />}
11     </div>
12   );
13 }
14 export default Parent;
```

7

useEffect () : Làm việc với Event resize

```

src > components > ResizeEventDemo.jsx > ...
1  import {useEffect} from "react";
2
3  export default function ResizeEventDemo() {
4    useEffect(() => {
5      // Hàm xử lý resize
6      const handleResize = () => {
7        if (window.innerWidth <= 800) {
8          // Biểu tượng nhấn window + .
9          console.log("★ Đạt kích thước <= 800px!");
10         }
11       };
12
13      // Đăng ký event
14      window.addEventListener("resize", handleResize);
15      console.log("Đã add event resize!");
16
17      // === Bỏ cleanup để demo hậu quả khi mount/unmount liên tục ===
18      return () => {
19        window.removeEventListener("resize", handleResize);
20        console.log("Đã remove event resize!");
21      };
22    }, []); // chỉ mount 1 lần
23
24    return (
25      <div>
26        <h3>Resize cửa sổ nhỏ hơn 800px để xem console!</h3>
27      </div>
28    );
29  }

```

*Clean up chỉ đơn giản là 1 hàm, khai báo trong return, bên trong useEffect(). React sẽ gọi hàm này khi **component bị unmount** - hoặc khi effect được chạy lại với dependency mới, đảm bảo clear được Interval tránh rò rỉ bộ nhớ*

Đã add event
resize!

4 ★ Đạt kích thước
<= 800px!

7 Đã add event
resize!

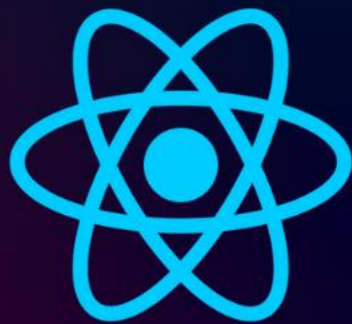
7

useEffect () : Làm việc với Event Keydown

```
src > components > KeydownEvent.jsx > ...
1  import {useEffect} from "react";
2
3  export default function KeydownEvent() {
4    useEffect(() => {
5      // Hàm xử lý resize
6      const handleKeyDown = (event) => {
7        console.log(event);
8        if (event.key) {
9          // Biểu tượng nhấn window + .
10         console.log("★ Phím được nhấn!");
11       }
12     };
13
14     // Đăng ký event
15     window.addEventListener("keydown", handleKeyDown);
16     console.log("Đã add event keydown!");
17
18     // === Bỏ cleanup để demo hậu quả khi mount/unmount liên tục ===
19     return () => {
20       window.removeEventListener("keydown", handleKeyDown);
21       console.log("Đã remove event keydown!");
22     };
23   }, []); // chỉ mount 1 lần
24
25   return (
26     <div>
27       <h3>Phím được nhấn !</h3>
28     </div>
29   );
30 }
```


[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 22

`useLayoutEffect()`

Hiểu rõ `useLayoutEffect`



<http://Tuhoc.CC>



No	Tiêu chí	useEffect	useLayoutEffect
1	Thời điểm chạy	Sau khi render và paint	Sau render, trước khi paint

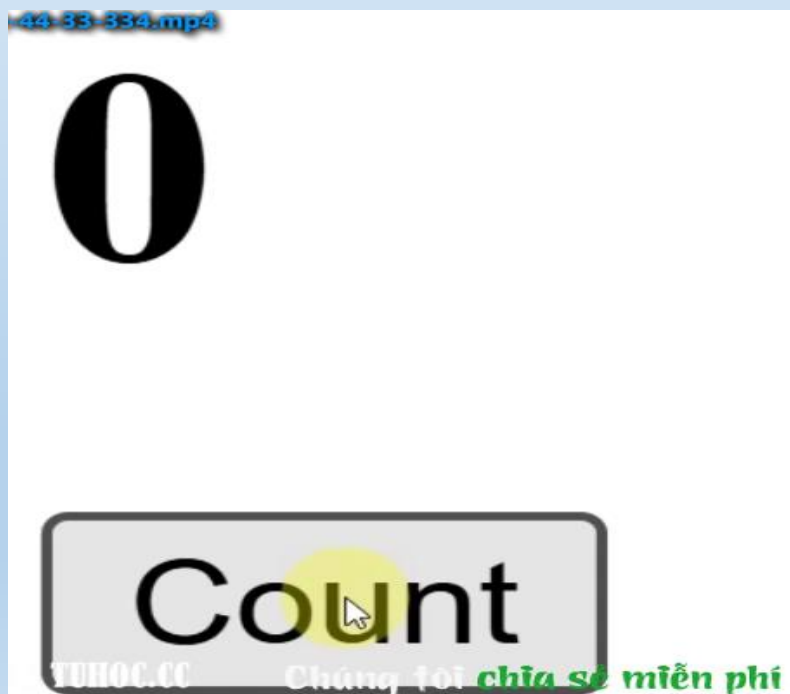
- ✓ **Render:** Hoàn thành quá trình thêm mới, hoặc cập nhật, xoá phần tử *DOM* → Tạo ra *DOM* hoàn chỉnh.
- ✓ **Paint:** Vẽ các phần tử *DOM* đó lên màn hình cho người dùng thấy.
- ❑ **Nhược điểm *useEffect()* :** Callback trong *useEffect* chạy sau paint. Nếu chỉnh sửa *DOM* hoặc layout tại đây sẽ làm giao diện phải vẽ lại lần nữa, gây nhấp nháy (*flicker*).

- ❑ *Nhược điểm `useEffect()` : Callback trong `useEffect` chạy sau paint. Nếu chỉnh sửa DOM hoặc layout tại đây sẽ làm giao diện phải vẽ lại lần nữa, gây nhấp nháy (flicker).*

```
App.jsx Demo.jsx 1 X
src > components > Demo.jsx > ...
1 import {useState, useEffect, useLayoutEffect} from "react";
2
3 export default function Demo() {
4   const [count, setCount] = useState(0);
5   //useEffect(() => {
6   useLayoutEffect(() => {
7     if (count > 2) {
8       setCount(0);
9     }
10  }, [count]);
11
12  const handleCount = () => {
13    setCount(count + 1);
14  };
15
16  return (
17    <div>
18      <h1>{count}</h1>
19      <button onClick={handleCount}>Count</button>
20    </div>
21  );
22 }
```

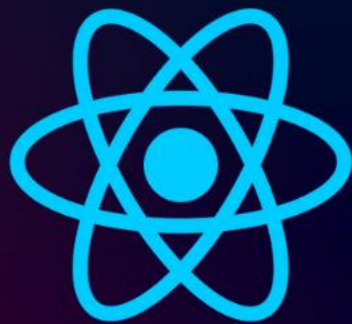
- ❑ *Nhược điểm useEffect() : Callback trong useEffect chạy sau paint. Nếu chỉnh sửa DOM hoặc layout tại đây sẽ làm giao diện phải vẽ lại lần nữa, gây nhấp nháy (flicker).*

No	Tiêu chí	useEffect	useLayoutEffect
1	Thời điểm chạy	Sau khi render và paint	Sau render, trước khi paint



[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 23

React.memo ()



<http://Tuhoc.CC>



1

React.memo()

Không dùng React.memo:

Cha render lại → Con cũng render lại, dù không cần thiết.

```
Demo.jsx x
src > components > Demo.jsx > ...
1  import { useState } from "react";
2
3  // Component con
4  function Child({ value }) {
5    console.log("Child render"); // Theo dõi lần render
6    return <div>Giá trị từ cha: {value}</div>;
7  }
8
9  // Component cha
10 function Parent() {
11   const [count, setCount] = useState(0);
12
13   return (
14     <div>
15       <button onClick={() => setCount(count + 1)}>Tăng Cha</button>
16       <div>Giá trị đếm của cha: {count}</div>
17       <Child value={1} />
18     </div>
19   );
20 }
21
22 export default Parent;
```

Tăng Cha

Giá trị đếm của cha: 6

Giá trị từ cha: 1

Child render

6 Child render

Mặc định, khi component cha render lại, toàn bộ component con cũng render lại, dù props không đổi. Điều này gây lãng phí tài nguyên, đặc biệt với các component con phức tạp, dẫn đến ứng dụng chậm, giật lag.

1

React.memo()

- ✓ Là một Higher Order Component (HOC) – hay có thể tạm hiểu là một phương thức (function) do thư viện React cung cấp.
- ✓ cách dùng của HOC là “bọc” (wrap) một component khác lại

React.memo() - kiểm soát việc render lại component con

- ✓ Props truyền vào con không thay đổi → Con sẽ **KHÔNG** render lại
- ✓ Props truyền vào con thay đổi → Con sẽ render lại.

Tăng Cha

Giá trị đếm của cha: 5

Giá trị từ cha: 1

Child render

```
import {useState} from "react";

// Component con
function Child({value}) {
  console.log("Child render"); // Theo dõi lần render
  return <div>Giá trị từ cha: {value}</div>;
}

// Component cha
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Tăng Cha</button>
      <div>Giá trị đếm của cha: {count}</div>
      <Child value={1} />
    </div>
  );
}

export default Parent;
```

```
import {useState, memo} from "react";

// Component con
const Child = memo(function Child({value}) {
  console.log("Child render"); // Theo dõi lần render
  return <div>Giá trị từ cha: {value}</div>;
});

// Component cha
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Tăng Cha</button>
      <div>Giá trị đếm của cha: {count}</div>
      <Child value={1} />
    </div>
  );
}

export default Parent;
```

Bước 1: Import memo

Bước 2: Bọc component con bằng memo

1

React.memo()

Hướng dẫn bọc memo, nếu component con viết riêng ở 1 file jsx

```
Child.jsx
import {memo} from "react";
// Component con
function Child({value}) {
  console.log("Child render"); // Theo dõi lần render
  return <div>Giá trị từ cha: {value}</div>;
}

export default memo(Child);
```

Bước 1: Import memo

Bước 2: Bọc component con bằng memo

```
Demo.jsx 1 Child.jsx
src > components > Child.jsx > ...
1 import {memo} from "react";
2 // Component con
3 export default memo(function Child({value}) {
4   console.log("Child render"); // Theo dõi lần render
5   return <div>Giá trị từ cha: {value}</div>;
6 });
```

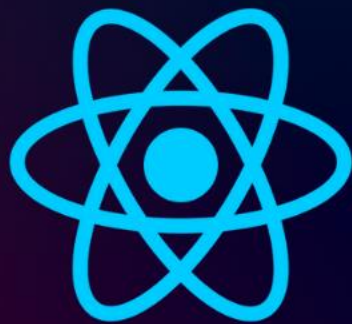
```
Demo.jsx Child.jsx
src > components > Demo.jsx > ...
1 import {useState} from "react";
2 import Child from "../Child.jsx";
3
4 // Component cha
5 function Parent() {
6   const [count, setCount] = useState(0);
7
8   return (
9     <div>
10       <button onClick={() => setCount(count + 1)}>Tăng Cha</button>
11       <div>Giá trị đếm của cha: {count}</div>
12       <Child value={1} />
13     </div>
14   );
15 }
16
17 export default Parent;
```

Chỉ nên dùng React.memo cho component con phức tạp, render chậm và props ít thay đổi.

Nếu props thay đổi liên tục, React.memo sẽ tốn thời gian so sánh, thậm chí còn chậm hơn so với việc render lại component con.

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 24

`useCallback ( )`

Cách Giảm Số Lần Render Component Con Hiệu Quả



<http://Tuhoc.CC>

1

Nêu vấn đề

```

src > components > Child.jsx > ...
1  import {memo} from "react";
2
3  function Child({value, onClick}) {
4    console.log("Child render"); // Xem lần render lại
5    return (
6      <div>
7        <div>Giá trị từ cha: {value}</div>
8        <button onClick={onClick}>Tăng Cha</button>
9      </div>
10   );
11 }
12
13 export default memo(Child);

```

Giá trị đếm của cha: 4
 Giá trị từ cha: 1
 Tăng Cha



Child render
 4 Child render

```

src > components > Demo.jsx > ...
1  import React, {useState} from "react";
2  import Child from "../Child.jsx";
3
4  function Parent() {
5    const [count, setCount] = useState(0);
6
7    const handleIncrease = () => {
8      setCount((prev) => prev + 1);
9    };
10
11   return (
12     <div>
13       <div>Giá trị đếm của cha: {count}</div>
14       <Child value={1} onClick={handleIncrease} />
15     </div>
16   );
17 }
18
19 export default Parent;

```

- **React.memo** chỉ kiểm tra **props** có thay đổi không.
- Khi cha render lại, nếu khai báo hàm callback trực tiếp trong hàm cha (không dùng `useCallback`):
 - + Mỗi lần render, hàm callback mới được tạo ra (khác ô nhớ)
- Do đó, props **onClick** truyền xuống con luôn thay đổi.
- Kết quả:
 - + Component con vẫn bị render lại, dù thực chất không cần thiết.

2

Cách dùng useCallback()

```
const yourFunctionName = useCallback(callbackFunction, [dependencies]);
```

```

Demo.jsx x Child.jsx
src > components > Demo.jsx > ...
1 import React, {useState, useCallback} from "react";
2 import Child from "../Child.jsx";
3
4 function Parent() {
5   const [count, setCount] = useState(0);
6
7   // const handleIncrease = () => {
8   //   setCount((prev) => prev + 1);
9   // };
10
11   // useCallback
12   const handleIncrease = useCallback((() => {
13     setCount((prev) => prev + 1);
14   }, []); // dependencies mảng rỗng
15   ↑
16   return (
17     <div>
18       <div>Giá trị đếm của cha: {count}</div>
19       <Child value={1} onClick={handleIncrease} />
20     </div>
21   );
22 }
23
24 export default Parent;

```

callbackFunction: Hàm callback cần ghi nhớ.
[dependencies]: Mảng phụ thuộc, **giống useEffect**.
 Khi giá trị nào trong mảng này đổi, callback mới sẽ được tạo lại.
 Nếu để mảng rỗng [], hàm chỉ tạo một lần duy nhất (từ đầu đến cuối component).

Giá trị đếm của cha: 9

Giá trị từ cha: 1

Tăng Cha

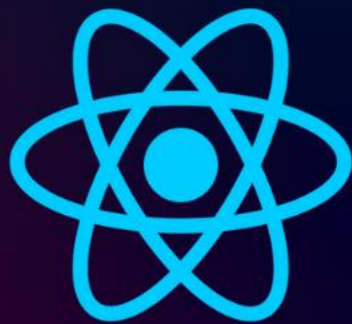


Child render

Child render 1 lần

[http://](http://Tuhoc.cc)

Tuhoc.cc



REACT JS



BÀI 25

`useMemo ()`

tối ưu component với
phép tính phức tạp



<http://Tuhoc.CC>



1

Nêu vấn đề

```

1 import Demo from "../components/Demo.jsx";
2
3 function App() {
4   return (
5     <>
6       <Demo number={1} />
7     </>
8   );
9 }
10
11 export default App;

```

Kết quả tính toán: 1000

Tăng đếm: 4



Tính toán lại...

4 Tính toán lại...

```

Demo.jsx x App.jsx
src > components > Demo.jsx > Demo
1 import {useState} from "react";
2 export default function Demo({number}) {
3   // Hàm tính toán nặng
4   function expensiveCalculation(num) {
5     console.log("Tính toán lại...");
6     let result = 0;
7     for (let i = 0; i <= 999; i++) {
8       result += num;
9     }
10    return result;
11  }
12
13  const result = expensiveCalculation(number);
14
15  const [count, setCount] = useState(0);
16
17  return (
18    <div>
19      <div>Kết quả tính toán: {result}</div>
20      <button onClick={() => setCount(count + 1)}>Tăng đếm: {count}</button>
21    </div>
22  );
23 }

```

Mỗi lần **component render lại**, **hàm tính toán nặng đều chạy lại**, dù **dữ liệu đầu vào (number) không đổi**.

🔊 Điều này gây lãng phí hiệu năng, đặc biệt khi phép tính phức tạp.

2

useMemo ()

```
const nameVariable = useMemo(computeFunction, [dependencies]);
```

computeFunction: Hàm trả về giá trị cần "ghi nhớ".

[dependencies]: Mảng phụ thuộc, giống *useEffect*. Khi đổi, giá trị mới sẽ được tính

Kết quả tính toán: 1000 → Tính toán lại...
Tăng đếm: 10

Child tính 1 lần nếu number không đổi

Sơ sánh giữ memo và useMemo:

- ✓ **React.memo:** Ghi nhớ component → Tránh render lại khi props không đổi.
- ✓ **useMemo:** Ghi nhớ giá trị → Tránh tính toán lại khi dependencies không đổi.

```

src > components > Demo.jsx > ...
1  import {useState, useMemo} from "react";
2
3  export default function Demo({number}) {
4    // Hàm tính toán nặng
5    function expensiveCalculation(num) {
6      console.log("Tính toán lại...");
7      let result = 0;
8      for (let i = 0; i <= 999; i++) {
9        result += num;
10     }
11     return result;
12   }
13
14   // Ghi nhớ kết quả tính toán, chỉ tính lại khi number thay đổi
15   const result = useMemo(() => expensiveCalculation(number), [number]);
16
17   const [count, setCount] = useState(0);
18
19   return (
20     <div>
21       <div>Kết quả tính toán: {result}</div>
22       <button onClick={() => setCount(count + 1)}>Tăng đếm: {count}</button>
23     </div>
24   );
25 }

```